Java Comparator interface

If we need to sort an array

```
String[] array = new String[] { "a", "b", "c" };

Arrays.sort(array);
```

Similarly,

```
Arrays.sort(intArr); // sort a numeric arry
Arrays.sort(cArr);  // sort a char array
```

If we need to sort an ArrayList/List

```
List<String> list = Arrays.asList(array);

Collections.sort(list);
```

If we need to sort an ArrayList of Objects, we use Comparable or Comparator interface :

```
List<Student> students = new ArrayList<>();

students.add(new Student(23, "Ram"));
students.add(new Student(35, "Shyam"));
students.add(new Student(83, "Aman"));
students.add(new Student(13, "Rohit"));
students.add(new Student(65, "Anuj"));

Collections.sort(students);

students.forEach(System.out::println);
```

Then, for that implement the Comparable interface :

```
public class Student implements Comparable<Student>{
    int marks;
    String name;

    public Student(int marks, String name){
        super();
        this.marks = marks;
        this.name = name;
    }

    public String toString(){
        return "Student [marks=" + marks + ", name=" + name + "]";
    }

    public int getMarks(){
        return marks;
    }

    public void setMarks(int marks){
        this.marks = marks;
    }

    public String getName(){
        return name;
    }

    public void setName(String name){
        this.name = name;
    }
```

```
    // -1 :  current object is bigger
    //  0 :  both objects are equal
    //  1 :  current object is smaller
    @Override
    public int compareTo(Student o){
        if(this.marks > o.marks) return 1;
        else if(this.marks < o.marks) return -1;
        return 0;
        //return this.name.compareTo(o.name);
    }

}
```

If you do not want to implement the Comparable interface, you can use a Comparator for the same :

```
Collections.sort(students, new Comparator<Student>() {
                                @Override
                                public int compare(Student o1, Student o2) {
                                    if(o1.marks > o2.marks) return 1;
                                    else if(o1.marks < o2.marks) return -1;
                                    return 0;
                                    //return o1.name.compareTo(o2.name);
                                }
                            });
```

The same can be done in java 8 as given below :

```
Collections.sort(students, (o1, o2) ->{
                                if(o1.marks > o2.marks) return 1;
                                else if(o1.marks < o2.marks) return -1;
                                return 0;
                            });
```

If it were the string comparision, the same can be simplified as given below, String internally implements the comparable interface :

```
Collections.sort(students, (o1, o2) -> o1.name.compareTo(o2.name));
```

This can be further simplified as

```
Collections.sort(students, Comparator.comparing(Student::getName));
```

If two students have the same marks, you would like to sort them by name as well. This can be done as :

```
Collections.sort(students, Comparator.comparing(Student::getMarks)
                                    .thenComparing(Student::getName));
```

To reverse the collection :

```
Collections.sort(students, Comparator.comparing(Student::getMarks)
                                    .thenComparing(Student::getName).reversed());
```

Sorting Date :

```
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Arrays;
import java.util.Comparator;

public class DateSort
{
        public static void main(String [] args)
        {
                String dates[] = {"05-11-2014","23-08-2014","05-02-2013"};

                Arrays.sort(dates, new Comparator<String>()
                {
                        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");

                        @Override
                        public int compare(String s1, String s2)
                        {
                                int result = -1;

                                try {
                                        result = sdf.parse(s1).compareTo(sdf.parse(s2));
                                } catch (ParseException ex) {
                                        ex.printStackTrace();
                                }

                                return result;
                        }
                });

                for (String date : dates)
                {
                        System.out.println(date);
                }
        }
}
```

Output :

```
05-02-2013

23-08-2014

05-11-2014
```